Sécurisation d'une application de messagerie instantanée
Projet de cryptographie
2025-2026

I – Introduction

Ce projet vise à sécuriser une application de messagerie instantanée en Java en mettant en œuvre progressivement des mécanismes cryptographiques assurant un niveau de sécurité de 128 bits. L'application comprend un serveur (*Server.java*) et des clients (*Client.java*) simulés sur un même PC, avec un script start.bat pour lancer un serveur et deux clients. Le fichier *Interceptor.java* contient des méthodes appelées lors du handshake, avant l'envoi et après la réception des messages, initialement utilisant rot13 pour simuler un chiffrement. Pour tester les vulnérabilités, notamment les attaques de type Man-in-the-Middle, le serveur utilise la méthode ServerInterceptor.onMessageRelay lors du relais des messages, permettant d'évaluer l'efficacité des protections mises en place.

II - Le Projet

1)Test de l'application initiale

Dans la classe *ServerInterceptor* on ajoute une méthode rot13 permettant de déchiffrer le message. On appelle cette méthode et on affiche le résultat, on retombe bien sur le message émis par le client

Server :

```
Server is now relaying messages between clients...
[Client 2 -> Server]: Fnyhg
Relaying from 2 to client 1 : Message chiffré Fnyhg  | Message déchiffré: Salut
[Server -> Client 1]: Fnyhg
```

Client :

```
Salut
[Interceptor] Encrypting message: Salut
You: Salut
```

2)Ajout du chiffrement symétrique

Sécurisation :
On récupère le mot de passe en argument, on le met en paramètre des constructeurs. On crée un constructeur coté Interceptor qui récupère le mot de passe et on dérive une clé AES a partir de ce mot de passe.

AES-128 offre un niveau de sécurité de 128 bits, considéré comme sûr contre les attaques brute force avec les technologies actuelles, on utilise donc ce niveau de sécurité comme demandé.

On utilise un **SHA-256** pour la méthode de hachage afin d'avoir une sécurité de 128 bits.

Ensuite on crée une méthode *processAES* qui prend en paramètre la clé et qui va être utilisé pour chiffrer et déchiffrer les messages en **AES CBC**. Cette méthode gère aussi l'**encodage en base64** pour la visualisation a l'écran.

Attaque MITM:
Je suis actuellement dans un mode de chiffrement CBC (Cipher Bloc Chaining) qui est un mode de chiffrement par bloc, chaque blocs de plain texte est XORé avec le bloc chiffré précédent, ce qui fait que si on modifie un bloc chiffré ça va impacter, le message en claire lors du déchiffrement. CBC ne garantit donc pas l'intégrité.

Si on teste avec un mode de chiffrement par flux comme AES-CTR on se rend compte que l'intégrité n'est pas protège non plus (pas de propagation d'erreurs).

Ex : Après une modification MITM d'un bit dans un chiffrement sans padding AES-CTR on voit que on peut modifier le message sans provoquer d'erreurs.

```
You: Hello Bob
```

```
[Interceptor] Decrypting message...
Other: Iello Bob
```

Le **padding** permet de compléter le dernier bloc a la fin d'un message pour que sa taille corresponde a la taille du bloc. Si l'altération MITM du message touche le dernier bloc. Le padding sera cassé et une erreur sera alors provoqué.

```
Other: [Decryption failed: AES processing failed]
```

Cependant si le dernier bloc n'est pas touché l'altération va passer inaperçu comme dans les screens précédents. Le padding n'est donc pas une solution d'intégrité.

Pour garantir l'intégrité il faut opter pour un **mode de chiffrement authentifié** comme **AES-GCM.** Les modes CBC, ECB, ou CTR sont vulnérables a des attaques d'altérations.

3)Ajout de l'intégrité des messages

Sécurisation :
On utilise a présent le mode **AES-GCM** qui garantit **confidentialité + intégrité + authenticité** des messages. L'intégrité est garantie par un mécanisme de tag. Si le tag calculé au déchiffrement est différent de celui envoyé, une erreurs comme celle si dessous est retournée.

AES-GCM Utilise AES en mode CTR pour chiffrer, c'est donc un chiffrement par flux. Le chiffrement est un XOR du texte en claire et du keystream. Le keystream dépend de la clé et d'un vecteur d'initialisation de **96 bits** (recommandé pour AES-GCM). Si 2 messages en claire sont chiffrés on aura 2 messages chiffrés différents. L'**unicité du chiffrement** est garantie

Attaque MITM:
Échec de l'attaque, altération détectée.

```
Other: [Decryption failed: AES-GCM processing failed: Tag mismatch]
```

4)Ajout de l'établissement de clé de session

Sécurisation :
Maintenant on change le mode de fabrication de la clé AES. On utilise la cryptographie asymétrique via ECDH pour obtenir une clé partagé sans avoir besoin d'échanger un secret en amont comme c'était le cas avant. Une **courbe elliptique de 256 bits (secp256r1)** est utilisé pour garantir un niveau de sécurité de 128 bits.
L'établissement de la clé de session se fait en plusieurs étapes a l'intérieur du handshake : La génération d'une paire de clé ECDH, l'envoie de la clé publique (encodé en base64), la réception de clé publique distante, le calcul du secret partagé et la dérivation de la clé AES-128 via **SHA-256** (pour garantir une sécurité de 128bits).
L'établissement de clé avec courbe elliptique Diffie-Hellman permet d'avoir une nouvelle clé pour chaque connexion et de garantir la **Perfect Forward Secrecy** (ne pas pouvoir déchiffrer les sessions passés avec la clé actuelle) tout en ayant de bonnes performances.

Attaque MITM:

Nous avons actuellement un mode de chiffrement authentifié (AES-GCM) garantissant l'authenticité, l'intégrité et la confidentialité. La faille de sécurité n'est donc pas ici. Nous avons cependant un échange de clé fait par ECDH éphémère, lors du handshake. Il n'y a ici aucun système d'authentification car aucun système de **signature ou de certificats**. L'authenticité n'est pas respectée, une attaque est donc possible.

Un attaquant MITM peut alors se placer entre le Client 1 et le Client 2 pour intercepter le handshake et replacer lors de l'échanges des clés publique, les clés des clients par sa propre clé publique. Le secret partagé serait alors établis entre l'attaquant et chacun des clients, ce qui lui permettrai de déchiffrer, lire, modifier tous les messages.

Pour implémenter cette attaque une nouvelle méthode ***performHandshake*** a été crée au sein de *ServerInterceptor.* Le code *Server.java* a du être légèrement modifier pour appeler cette méthode afin de différencier l'interception du handshake et l'interception des messages chiffrés (fait par la méthode ***onMessagerelay***).
Coté Server.java la méthode *run()* est modifié pour envoyé la clé publique reçue au *ServerInterceptor* lors du handshake. Ceci est fait avant l'appel a la méthode broadcastMessage qui va appeler onMessagerelay pour l'interception des messages.
Coté ServerInterceptor.java, la méthode *performhandshake* va s'occuper de générer une paire de clé MITM, décoder la clé publique du client base64, calculer le secret partagé entre le MITM et le client, dériver la clé AES et la stocker. La fonction retourne ensuite la clé publique de l'attaquant qui va donc remplacer celle du client légitime.
Ensuite, la méthode *onRelayMessage* va intercepter chaque message chiffrés et va pouvoir les déchiffrer le message via la clé dérivé du secret partagé entre le client et le MITM et re chiffrer pour relayer le message potentiellement modifie vers le réel destinataire.

Client 1 :

```
salut
[Interceptor] Encrypting message: salut
You: salut
```

Client 2 :

```
[Interceptor] Decrypting message...
Other: salut
```

ServerInterceptor:

```
[Client 1 -> Server]: MYHhWalpKMTnxBUrrWp2PON9FTUYookNNCOlBYZH3lxb
[MITM] Intercepted from client 1: salut
[Server -> Client 2]: 4oux0/kSBq2xEk8PYc9gwjAw0DZIbCPQPPzqz2fwOw7q
```

5)Ajout de la signature des clés publiques ECDH

Sécurisation

Nous allons a présent utiliser openSSL pour générer une paire de clé ECDSA (Elliptique Curve Digital Signature Algorithme) long terme pour chaque client. Ce type de clés va permettre d'utiliser un algorithme de signature pour pouvoir authentifier le message.

Voici la commande pour la **clé publique** (client 1) :

openssl ecparam -name prime256v1 -genkey -noout -out client1_private.pem

Voici la commande pour la **clé privé** (client 1) :

openssl ec -in client1_private.pem -pubout -out client1_public.pem

Voici la commande pour verifier une clé (courbe, clé priv/pub) :

openssl ec -in client1_private.pem -text -noout

On a donc une clé publique et privé pour chaque clients. On adapte le code pour pouvoir passer en paramètre du client la clé publique et privé.

Maintenant, nous allons modifier la phase de handshake pour **signer les clés publiques éphémères ECDH avec la clé longue durée ECDSA** afin de pouvoir afin d'authentifier le client pour éviter les attaques MITM d'un remplacement de la clé publique par celle d'un attaquant.
Dans notre nouvelle paire de clés, la clé privé est utilisée pour signer un message et la clé publique est utilisé pour vérifier la signature.
On modifie alors la fonction *onHandShake* pour ajouter la signature (le calcul du hash de la clé sur lequel la signature est effectué est fait en interne de ECDSA).

Attaque MITM :

A présent on doit s'occuper du mécanisme de vérification de la signature. Pour cela il nous faut la clé publique ECDSA du pair. Or on ne peut pas la demander via le handshake car l'envoie de cette clé de serait lui même pas authentifié, on ne pourrait alors pas savoir si quelqu'un a falsifié la clé en MITM (comme précédemment). On se rend alors compte que sans autorité de confiance, on ne peut pas se partager la clé permettant de vérifier l'authenticité du message sans risquer une attaque MITM.
Pour cette partie nous avons passé en paramètre la clé publique ECDSA (présente en local) pour faire fonctionner la vérification de signature pour faire échouer les attaques MITM. (solution temporaire)

6)Certification des clés

Sécurisation

Une autorité de certification a été générée à l'aide d'OpenSSL. Une paire de clés ECDSA utilisant la courbe prime256v1 (secp256r1) a été créée afin d'atteindre un niveau de sécurité d'environ 128 bits.

openssl ecparam -name prime256v1 -genkey -noout -out ca_private.pem

On crée le certificat autosigné X509 :

openssl req -new -x509 -key ca_private.pem -out ca_cert.pem -days 3650

On crée mainteant la demande de certificat pour chaques clés publique clients :

openssl req -new -key client1_private.pem -out client1.csr

On a donc une **CSR (Certificat Signing Request)** qui lie la clé publique a l'identité du client.

On utilise maintenant la clé privé du CA pour signer les certificats clients :

openssl x509 -req -in client1.csr -CA ca_cert.pem -CAkey ca_private.pem -CAcreateserial -out client1_cert.pem -days 365

On utilise le certificats du CA pour vérifier la signature du certificat client comme ci dessous :

```
ssagerie_Secure/CryptoChat$ openssl verify -CAfile ca_cert.pem client2_cert.pem
client2_cert.pem: OK
```

Pour récapituler on garde les clés suivantes :

- ***ca_private.pem***
  clé privée de l'AC (sert à signer les certificats)
- ***ca_cert.pem***
  certificat de l'AC → utilisé par les clients pour **vérifier les certificats**
- ***client1_private.pem***
  clé privée de client1 → pour **signer / prouver son identité**
- ***client1_cert.pem***
  certificat signé par l'AC → envoyé pendant le **handshake**
- ***client2_private.pem***
  clé privée de client2

- ***client2_cert.pem***
  certificat signé par l'AC

On passe maintenant au démarrage d'un client son certificat, sa clé privé et le certificat racine du CA. Puis on modifie la fonction onHandshake pour intégrer un mécanisme d'authentification basé sur les **certificats X.509**.
Concrètement, lors du **handshake**, chaque client envoie désormais son certificat, sa clé publique ECDH éphémère ainsi que la signature ECDSA de cette clé réalisée avec sa clé privée longue durée. Le client distant reçoit ensuite ces informations et commence par vérifier le certificat reçu à l'aide du certificat racine de la CA, afin de s'assurer que la clé publique longue durée appartient bien à un client légitime. Il vérifie également la validité temporelle du certificat.

Une fois le certificat validé, la clé publique longue durée contenue dans ce certificat est utilisée pour vérifier la signature de la clé publique ECDH éphémère. Cela garantit que la clé utilisée pour l'échange Diffie-Hellman provient bien du propriétaire du certificat et qu'elle n'a pas été modifiée.

Après ces vérifications, les deux clients peuvent calculer le secret **partagé ECDH**, qui est ensuite haché avec **SHA-256** afin de dériver une clé de session **AES-128** utilisée pour chiffrer les messages avec **AES-GCM**.

Attaque MITM

Si on tente une attaque en remplaçant la clé du client mais en ne signant pas correctement on voit que l'attaque est bien détecté coté client et retourne une erreur :

(Pour faire fonctionner l'attaque dans mon code il faut dé commenter l'appel au MITM lors du handshake dans run() (Server.java).)

```
--- Handshake Phase ---
[Interceptor] Starting ECDH handshake
[Interceptor] Peer certificate verified
[Interceptor] ALERT: Possible MITM detected! Signature verification failed.
Connection error: ECDH handshake failed
java.io.IOException: ECDH handshake failed
```

7)Attaques résiduelles

Même après avoir implémenté un protocole ECDH pour l'échange de clés éphémères, combiné à AES-GCM pour la confidentialité et l'intégrité, certaines attaques restent possibles :

Rejeu de message
Un attaquant peut capturer un message chiffré sur le réseau et le renvoyer ultérieurement au destinataire. Comme chaque message est chiffré et authentifié via AES-GCM, l'intégrité est vérifiée, mais rien n'empêche qu'un message valide soit réexpédié. Cela peut provoquer la duplication de messages ou des comportements inattendus dans l'application.

Suppression ou blocage de message
Un attaquant MITM peut choisir de bloquer certains messages ou de les supprimer avant qu'ils n'atteignent le destinataire. L'intégrité cryptographique ne protège pas contre la perte de messages, et le protocole actuel ne fournit aucun mécanisme pour détecter qu'un message a été supprimé ou perdu.

Solution de sécurité

Pour éviter les attaques de type rejeu ou suppression de message, il est nécessaire d'introduire un mécanisme de traçabilité des messages, permettant au destinataire de détecter si un message est ancien, répété ou manquant.

Une méthode simple serait d'introduire des numéros de séquences a chaque messages. Chaque client inclus un compteur qui s'incrémente avant chaque envoi. Le destinataire garde en mémoire le dernier numéro de séquence reçu et si lors de la réception le numéro de séquence est inférieur ou égal au dernière reçu le message est rejeté. De même le numéro de séquence saute des nombres.

Dans mon code le compteur est gérer dans les méthodes *beforeSend* et *afterReceive* de *Interceptor.java*. Le compteur est initialisé a 0 lors du handshake grace a la méthode *initCounters().*

Le message ci dessous est affiché si un rejeu est détecté.

```
Other: [Replay detected! Message discarded]
```

Perfect Forward Secrecy (PFS)

Si un attaquant enregistre tous les messages chiffrés puis obtient plus tard les clés privées des clients, il ne pourra pas déchiffrer les messages des sessions passées. Cela est dû au fait que chaque session utilise une clé AES éphémère dérivée d'un secret partagé ECDH différent pour chaque handshake. Même avec les clés privées à long terme, les anciens secrets de session ne peuvent pas être recalculés.

Cette propriété de sécurité s'appelle la **Perfect Forward Secrecy (PFS).** Elle garantit que la compromission des clés longues ne menace pas la confidentialité des communications passées.

III – Difficultés rencontrées

Plusieurs difficultés techniques ont été rencontrées lors de l'implémentation du protocole sécurisé.

La première difficulté a concerné la **mise en place du chiffrement AES-GCM**. Il a fallu comprendre comment gérer correctement le **vecteur d'initialisation (IV)** ainsi que le **tag d'authentification**. Lors des premiers tests, certaines erreurs comme *"Tag mismatch"* apparaissaient lors du déchiffrement, ce qui indiquait que l'intégrité du message ne pouvait pas être vérifiée. Ce problème a été résolu en utilisant un IV aléatoire de 96 bits, recommandé pour AES-GCM, et en concaténant correctement l'IV et le message chiffré avant l'encodage Base64 afin que le destinataire puisse reconstruire les paramètres nécessaires au déchiffrement.

Une autre difficulté importante a été la **gestion des clés et certificats au format PEM** générés avec OpenSSL. Java ne charge pas directement ce format, il a donc fallu lire les fichiers, supprimer les en-têtes PEM, décoder le contenu Base64 puis reconstruire les objets PrivateKey, PublicKey et X509Certificate. Ce problème a été résolu en utilisant les classes KeyFactory, PKCS8EncodedKeySpec et CertificateFactory, permettant de convertir correctement les données en objets cryptographiques utilisables par l'application.

La **vérification des signatures ECDSA des clés publiques ECDH** a également demandé plusieurs ajustements. Il fallait s'assurer que la signature était réalisée sur les octets exacts de la clé publique ECDH et que la vérification utilisait bien la clé publique longue durée contenue dans le certificat du client distant. Après correction de ces points, la vérification de signature fonctionne correctement et permet de détecter toute tentative de modification de la clé lors du handshake.

Enfin, l'**implémentation du mécanisme de protection contre les attaques de rejeu** a posé des problèmes de synchronisation. Initialement, les compteurs de séquence n'étaient pas correctement initialisés, ce qui provoquait le rejet de messages légitimes avec l'erreur *"Replay detected! Message discarded"*. Ce problème a été résolu en initialisant correctement les compteurs lors de l'établissement de la session et en incrémentant le compteur à chaque envoi de message. Le destinataire vérifie ensuite que le compteur reçu est strictement supérieur au précédent afin de détecter les duplications de messages.

IV – Améliorations possibles

Sécurisation du stockage local : Les clés privées et les messages stockés sur le client peuvent représenter un point de vulnérabilité en cas d'accès non autorisé à la machine. Un chiffrement fort des fichiers locaux, associé à un contrôle d'accès et éventuellement à une protection par mot de passe ou module matériel sécurisé (HSM ou Secure Element), renforcerait la sécurité globale.

Renforcement de l'authentification des clients : Bien que l'usage de certificats X.509 permette de vérifier l'identité des clients, il serait possible d'améliorer la sécurité en mettant en place une infrastructure à clés publiques complète (PKI) avec gestion des certificats compromis (révocation) et vérification en ligne. Cela limiterait le risque d'usurpation d'identité même si une clé privée venait à être exposée.

Amélioration des performances : La génération et la vérification des tags AES-GCM, ainsi que la gestion des vecteurs d'initialisation, peuvent devenir coûteuses si l'application est utilisée à grande échelle. Optimiser la gestion des IV, des clés et des threads pour le chiffrement/déchiffrement permettrait de réduire la latence tout en maintenant le niveau de sécurité pour une gestion multi utilisateurs.

IV – Conclusion

Ce projet a permis de mettre en place progressivement plusieurs mécanismes nécessaires à la sécurisation d'une application de messagerie. Initialement, l'application utilisait un chiffrement simple et non sécurisé. L'introduction d'AES-128 en mode GCM a permis d'assurer la confidentialité et l'intégrité des messages.

Un échange de clé via ECDH éphémère a ensuite été implémenté afin d'établir une clé de session partagée entre les clients. L'authentification des clés a été assurée grâce à des signatures ECDSA et à l'utilisation de certificats X.509, permettant de se protéger contre les attaques de type Man-in-the-Middle.

Enfin, l'ajout de numéros de séquence dans les messages permet de limiter les attaques par rejeu. Ce projet montre qu'un protocole sécurisé repose sur la combinaison de plusieurs mécanismes cryptographiques correctement intégrés.

V – Sources

→ Diapos de cours Cryptographie

→ Intelligences artificielles LLM

→ https://www.wikipedia.org/

VI – Archive

L'archive du développement du projet est disponible sur GitHub et accessible via le lien suivant :

https://github.com/CamilleBainm/Projet_Cryptographie_Messagerie_Secure